

AI Image Detection under CPU and Time Constraints

Architecture of Machine Learning Systems - SoSe 2026

Riccardo Tellarini - XiangYu Zhang - Thomas Bove

1. Introduction and Solution Overview

The rapid improvement of image generation models has made AI-generated content increasingly difficult to distinguish from real photographs. At the same time, an effective detector must not only achieve high classification accuracy, but also limit the number of real images incorrectly flagged as artificial. This project addresses the binary classification of images as `REAL` or `AI-GENERATED`, where the five synthetic sources of the provided dataset (SD 2.1, SDXL, SD 3, DALL-E 3, Midjourney) are merged into a single positive class. The objective is to maximize recall on AI-generated images while keeping the false-positive rate (FPR) on real images at or below 20%.

The setting imposes a strict systems contract: all scripts run CPU-only inside a Docker container without internet access, with per-script timeouts. These constraints shaped the solution: the pipeline is built around a CPU-efficient architecture, deadline-driven training with periodic atomic checkpointing, and automatic threshold calibration. After the initial systems-oriented development, we ran 30-trial automatic hyperparameter searches for both modeling tasks and evaluated the best configuration for each task across ten seeds.

The solution has four stages. (i) A deterministic cleaning step decodes and audits every training image, removes exact duplicates, and documents strong class-correlated geometry and encoding shortcuts. (ii) Training of a compact CNN from scratch with balanced class sampling under an explicit time budget, calibration of the decision threshold on the calibration split, and comparison against a second model family (a basic Random Forest on engineered features). (iii) Fine-tuning of the previous checkpoint with realistic image corruptions (JPEG recompression, blur, rescaling, noise) and conservative calibration on both the clean and the augmented calibration splits. (iv) Analysis of the final model with gradient saliency, signed occlusion sensitivity, and per-generator error analysis, treating the explanations critically.

The remainder of the report describes the dataset analysis, model development, robustness strategy, validation protocol, and explainability results in detail.

2. Task 1: Dataset Exploration and Cleaning

2.1 Dataset Composition and Descriptive Analysis

The training split initially contains 29 688 labeled images. The six original source classes are perfectly balanced, with 4 948 samples per class. However, merging the five synthetic sources into a single `AI-GENERATED` label produces a strongly imbalanced binary problem: 4 948 `REAL` images (16.670 %) and 24 740 `AI-GENERATED` images (83.330 %). Accuracy is therefore uninformative: always predicting “AI” already yields 83.3%. The imbalance is handled during training by balanced sampling rather than by discarding data during cleaning. All 29 688 images decode successfully as RGB JPEG files, so format and color mode carry no class information.

Geometry, in contrast, is a good class predictor. As shown in Figure 1, every AI-generated image is square and occurs at two resolutions: 320×320 pixels (19 818 images) or 270×270 pixels (4 922 images). Real images are more diverse, with widths from 156 to 640 pixels and heights from 128 to 640 pixels, including landscape, portrait, and square formats. Their median aspect ratio is 1.333, whereas the AI-generated aspect ratio is always exactly 1.0.

Encoded file size also appears to be class-correlated: the median is 48.7 kB for real versus 26.0 kB for AI-generated images. This difference is confounded by resolution, JPEG quality and scene complexity, and must not be read as intrinsic evidence of authenticity.

These are acquisition and preprocessing artifacts of how the dataset was assembled, not properties of image generation. A model with access to original geometry could achieve high performance without analysing semantic image content, but would likely fail on differently prepared data. We therefore do not filter by geometry or file size during cleaning, resize every image to a common square input so the model never observes original dimensions directly, and revisit possible residual shortcut behavior in Task 4. Still, resizing could not provably eliminate the shortcut because rectangular real images are distorted more strongly than natively square AI images.

2.2 Deterministic Cleaning Pipeline

The cleaning procedure in `clean.py` is intended to be conservative: it removes only invalid or redundant images. Parquet files are processed in order, while the original file names and row indices are retained. Each image is converted to bytes and fully decoded with PIL to identify unreadable or truncated files. The script also records image dimensions, aspect ratio, colour mode, format, and encoded file size for analysis.

Exact duplicates are identified using SHA-256 hashes of the encoded image bytes. For duplicate groups with the same label, only the first image in the processing order is kept. If identical image bytes occur with conflicting labels, the entire duplicate group is removed rather than assigning a label arbitrarily. Cleaning occurs before the internal train-validation split, preventing duplicate images from appearing in both sets and inflating validation performance.

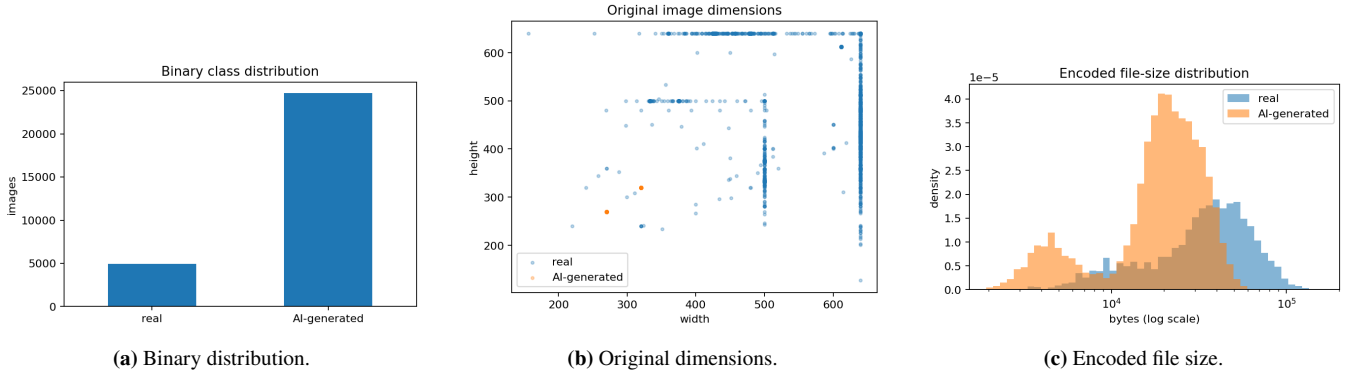


Figure 1: Training-set exploration before cleaning. The binary imbalance results from merging five equally sized synthetic source classes. In panel (b), heavy overplotting hides the frequency of the two AI-generated resolutions: the two orange locations represent 19 818 and 4 922 images rather than two individual samples. File size is shown on a logarithmic axis and is confounded by resolution and JPEG encoding.

All 29 688 images were fully decodable, and no duplicate groups with conflicting labels were found. The procedure identified 299 exact-duplicate groups and removed 312 redundant rows, retaining 29 376 images (98.950 % of the original set). The final cleaned dataset contains 4 887 real and 24 489 AI-generated images, so the class proportions remain essentially unchanged. Rather than duplicating the image bytes, `clean.py` writes a manifest containing the source parquet file, row index, binary label, keep flag, and removal reason.

Table 1: Deterministic cleaning results on `data/train/`.

Outcome	Count
Images (all fully decodable)	29 688
Exact-duplicate hash groups	299
Duplicate copies removed	312
Conflicting-label duplicate groups	0
Retained (4 887 real + 24 489 AI)	29 376

3. Task 2: Modeling and Tuning under Time Constraints

3.1 Data Preparation and Model Architecture

`prepare.py` consumes the cleaning manifest, decodes each retained image once, resizes it bilinearly to 96×96 RGB, and stores it as a `uint8` array. This removes repeated JPEG decoding and parquet access from the training loop.

The final model is a compact CNN trained from scratch. It comprises four convolutional stages containing seven convolution layers in total, followed by global average pooling and a linear layer producing one binary logit. The 2nd, 4th, and 6th convolutions use stride 2, reducing the spatial resolution from 96×96 to 48×48 , 24×24 , and 12×12 during feature extraction. With `channels=32`, the architecture has 435,297 trainable parameters and uses no dropout.

Table 2: Final SmallCNN architecture. Each convolution is 3×3 , has no bias, and is followed by BatchNorm and ReLU.

Stage	Operation	Stride	Output
Input	RGB image		$96 \times 96 \times 3$
Block 1a	conv $3 \rightarrow 32$	1	$96 \times 96 \times 32$
Block 1b	conv $32 \rightarrow 32$	2	$48 \times 48 \times 32$
Block 2a	conv $32 \rightarrow 64$	1	$48 \times 48 \times 64$
Block 2b	conv $64 \rightarrow 64$	2	$24 \times 24 \times 64$
Block 3a	conv $64 \rightarrow 128$	1	$24 \times 24 \times 128$
Block 3b	conv $128 \rightarrow 128$	2	$12 \times 12 \times 128$
Block 4	conv $128 \rightarrow 128$	1	$12 \times 12 \times 128$
Head	global avg. pool + linear		1 logit

3.2 Balanced, Deadline-Driven Training

Training uses balanced cycles containing every real training image and an equal-sized random subset of AI images. This gives both the optimizer and BatchNorm statistics equal class exposure despite the approximately 1:5 imbalance. AdamW and binary cross-entropy with logits are used, while checkpoints are selected by recall at the constrained internal operating point, with AUC as a tie-breaker.

The loop is step-based and stops according to elapsed wall-clock time. The tuned implementation uses a training budget of approximately 550 s, validates every 50 optimizer steps, and reserves time for final calibration, official validation, and artifact writing. Atomic checkpoints prevent a timeout from corrupting the saved model.

3.3 Automatic Hyperparameter Search

We used an automatic Optuna adaptive search with 30 trials. The search varied learning rate, batch size, internal validation fraction, scheduler length, calibration target FPR, channel width, and weight decay. Trial checkpoints were selected on the

internal holdout, while the calibration split determined the numerical threshold. Validation metrics were then used to compare trial configurations. Consequently, the reported validation values are development estimates rather than a completely untouched final estimate. The hidden holdout remains the definitive generalization test.

The selected configuration came from a focused rerun in the best-performing region:

Table 3: Selected Task 2 hyperparameters.

Learning rate	6.062×10^{-4}	Batch size	64
Internal validation fraction	0.1331	Scheduler cycles	15
Calibration target FPR	0.155	Channels	32
Weight decay	0	Seed	0

Across the 30 trials, calibration recall transferred well to validation (correlation 0.993), although validation recall was on average 1.34 percentage points lower. FPR transferred much less reliably (correlation 0.188): validation FPR was on average 2.65 points higher than calibration FPR, with individual deviations ranging from -3.16 to $+7.48$ points. Only four trials simultaneously achieved recall ≥ 0.8 and FPR ≤ 0.2 on validation. This confirms that a calibration target safely below 0.20 is necessary, but does not guarantee the same FPR on a separate finite sample.

3.4 Result Comparison with a Classical Model

To compare two model families, a Random Forest baseline was trained using 20 engineered features. These include RGB colour means and standard deviations, gradient statistics, chroma features, and an eight-bin radial Fourier profile. The classifier uses 300 trees, `class_weight="balanced"`, and a fixed random seed. Its decision threshold is calibrated using the same independent calibration procedure as the CNN.

The Random Forest achieves non-trivial performance, showing that global colour, edge, and frequency information is correlated with the class label. However, these features may also reflect acquisition or compression shortcuts and cannot represent spatial image patterns as flexibly as the CNN.

The final seed-0 run selected step 2300 of 2582, processed 164 488 training examples over 19 balanced cycles, and finished in 540.9 s. The automatically fitted threshold was 0.661539. Calibration achieved recall 0.8196 at FPR 0.1522; on validation, the same threshold achieved recall 0.8098 at FPR 0.1755 and ROC AUC 0.8864. The suggested Task 2 target is therefore reached while retaining a 2.45pp FPR margin.

Table 4: Task 2 comparison at automatically calibrated thresholds.

Split	Model	Recall	FPR	Precision	F1	AUC
Validation	SmallCNN	0.8098	0.1755	0.9583	0.8778	0.8864
Validation	Random Forest	0.5994	0.1649	0.9476	0.7343	0.7912
Augmented	SmallCNN	0.7364	0.2620	0.9337	0.8234	0.8097
Augmented	Random Forest	0.4578	0.2139	0.9147	0.6102	0.6765

The Random Forest uses 20 global color, gradient, chroma, and Fourier features. It remains a useful non-neural baseline, but the tuned CNN improves clean validation recall by 21.0 percentage points and AUC by 0.095.

3.5 Seed stability

The selected configuration was rerun with seeds 0-9. Validation recall ranged from 0.7692 to 0.8237 and FPR from 0.0904 to 0.2128; the means were 0.799 ± 0.015 and 0.168 ± 0.033 , respectively. Eight seeds respected the FPR constraint, five reached recall 0.8, and three satisfied both simultaneously. The target is therefore achieved by the packaged seed-0 run but is not fully seed-stable. Small changes in learned score distributions, stochastic balanced sampling, internal split composition, and checkpoint selection can move a quantile-calibrated threshold across several real validation examples. Since validation contains only 188 real images, one additional false positive changes the observed FPR by approximately 0.53 percentage points.

4. Task 3: Data Augmentation and Robustness

4.1 Robust Fine-Tuning and Augmentation

Task 3 initializes the same SmallCNN from the tuned Task 2 checkpoint and performs low-learning-rate robust fine-tuning. Each sample is transformed with probability 0.7648; when augmentation is triggered, one of four corruption families is sampled uniformly. The expected stream therefore contains approximately 23.5% clean images and 19.1% from each corruption family.

Table 5: Transformations used during robust fine-tuning.

Transformation	Range	Purpose
JPEG recompression	quality 15-75	Simulates aggressive recompression and blocking artifacts.
Gaussian blur	radius 1.0-4.5	Suppresses local sharpness and high-frequency evidence.
Resolution degradation	scale 0.15-0.6	Simulates strong downscaling followed by restoration to native resolution.
Gaussian noise	$\sigma = 8-30$	Simulates sensor, transmission, or editing noise.

Strong rotations, crops and color jitter were deliberately excluded from the corruption set, since they would alter the subtle generation and compression evidence the detector must rely on, rather than simulating realistic real-world degradation such as messaging-app re-uploads or resizing. The selected configuration uses an initial learning rate of 2.665×10^{-4} , cosine minimum learning rate 2.259×10^{-6} , batch size 96, internal validation fraction 0.1256, 13 scheduler cycles, augmentation probability 0.7648, weight decay 2.55×10^{-6} , and final calibration target FPR 0.1791.

4.2 Robust Calibration, Tuning, and Stability

Checkpoint selection prioritizes augmented recall, then clean recall, and finally mean clean/augmented AUC. Separate thresholds are estimated on clean and augmented calibration data, and the larger threshold is deployed. The selected run produced a clean-calibration threshold of 0.570797 and an augmented-calibration threshold of 0.613405, so the augmented threshold determined the final operating point.

Task 2 and Task 3 use different calibration targets (0.155 versus 0.1791) and different calibration domains (clean-only versus clean and augmented combined). Some of the operating-point differences reported below may therefore stem partly from calibration choices rather than from fine-tuning alone.

A second 30-trial Optuna study tuned the fine-tuning learning rates, batch size, validation fraction, scheduler length, augmentation probability, final calibration target, and weight decay. Augmented calibration recall correlated with augmented validation recall at 0.875, and validation recall was on average only 0.35 percentage points lower. FPR transfer remained less predictable (correlation 0.382), although its average shift was only +0.30 points. Fourteen of 30 trials satisfied both augmented recall ≥ 0.6 and FPR ≤ 0.2 .

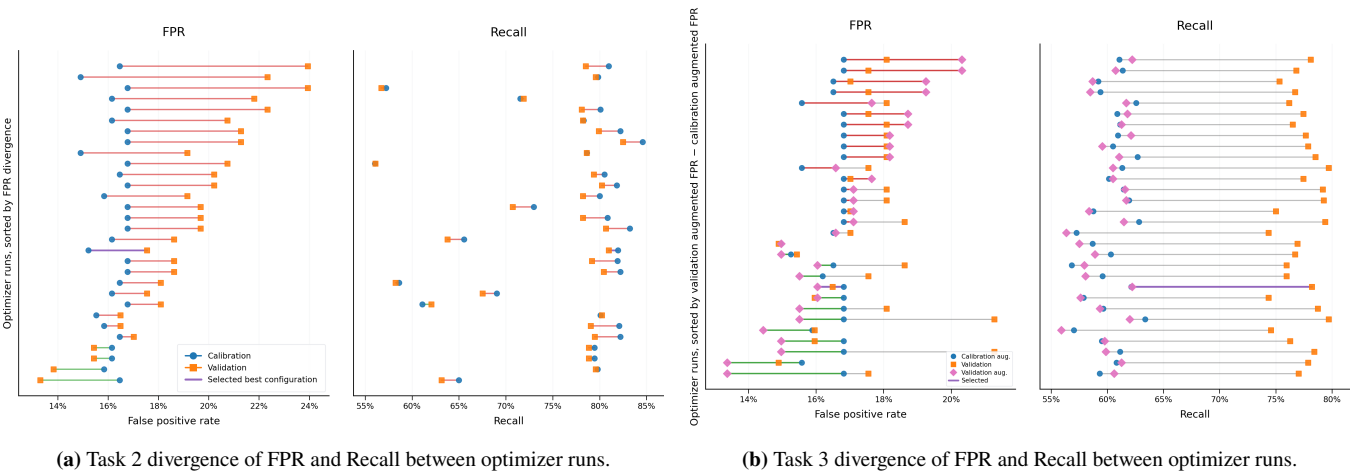


Figure 2: Divergence of FPR and Recall over 30 hyperparameter trials. This shows that in this model, calibration and validation sets’ parameters are not strongly correlated.

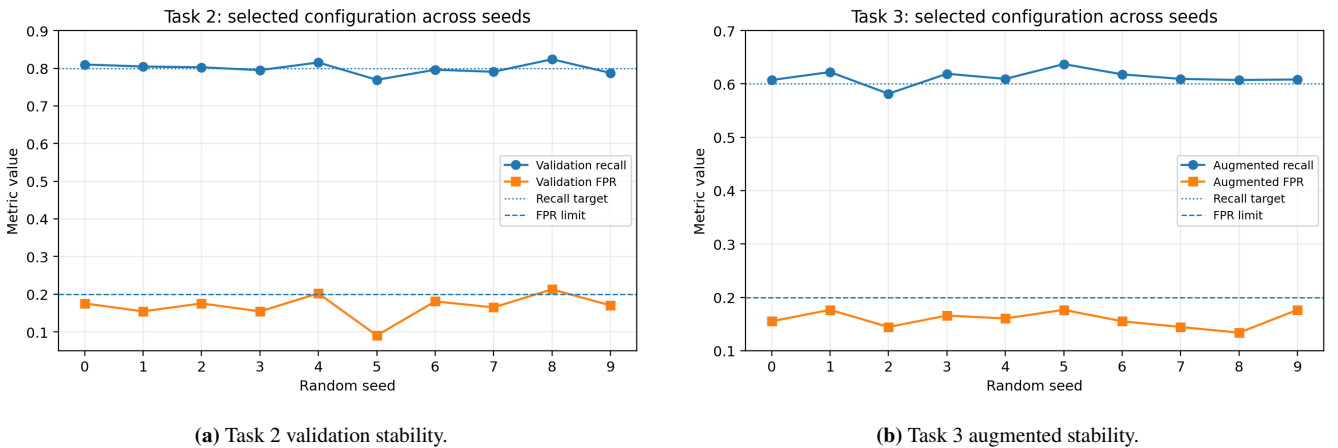


Figure 3: Recall and FPR for the selected hyperparameters across seeds 0–9. Task 2 is close to both boundaries and is visibly less stable; Task 3 is more stable on the augmented operating point.

Across seeds 0–9, augmented Task 3 recall ranged from 0.5816 to 0.6371 and FPR from 0.1337 to 0.1765, with means 0.612 ± 0.014 and 0.159 ± 0.015 . Nine seeds reached recall 0.6 and all respected the augmented FPR constraint. Clean FPR ranged from 0.1702 to 0.2128, so eight seeds respected the constraint on both domains. Compared with Task 2, the augmented target is therefore more stable, although clean-domain FPR remains sensitive to seed.

Two further limitations should be noted. First, the individual contributions of JPEG recompression, blur, resolution degradation, and Gaussian noise were not evaluated separately, so the observed robustness gain cannot be attributed to any single corruption family. Second, the internally augmented calibration and validation views remain only an approximation of the officially supplied `validation_augmented` distribution; any residual mismatch between the two augmentation pipelines could contribute to the imperfect transfer documented above.

4.3 Final Results and Interpretation

The final seed-0 run completed 1093 steps, processed 103 800 examples over 12 balanced cycles, selected the final step, which suggests training remained limited by the available wall-clock budget rather than exhibiting clear overfitting on the internal validation criterion, and finished in 554.4 s. For comparison, Task 2 completed 2582 steps in a similar wall-clock budget; Task 3’s lower step count reflects the added cost of on-the-fly corruption and dual (clean and augmented) internal validation passes each time a checkpoint is evaluated. It achieved augmented calibration recall 0.6145 at FPR 0.1682. On official validation, clean recall was 0.7831 at FPR 0.1702; on augmented validation, recall was 0.6073 at FPR 0.1551. The Task 3 recall target of 0.6 is therefore reached while the hard FPR constraint is satisfied on both domains.

Table 6: Final Task 2 and Task 3 operating points.

Split	Model	Recall	FPR	Precision	F1	AUC
Validation	Task 2	0.8098	0.1755	0.9583	0.8778	0.8864
Validation	Task 3	0.7831	0.1702	0.9582	0.8618	0.8831
Augmented	Task 2	0.7364	0.2620	0.9337	0.8234	0.8097
Augmented	Task 3	0.6073	0.1551	0.9515	0.7414	0.7909

Fine-tuning does not improve augmented recall or AUC relative to Task 2. Instead, it shifts the detector to a substantially safer operating point: augmented FPR falls by 10.7 percentage points, while recall remains just above the requested target. The strongest demonstrated contribution of Task 3 is therefore robust constraint compliance rather than improved threshold-independent separability.

The remaining instability can be explained by the combination of stochastic training, finite calibration and validation samples, and threshold estimation near a hard constraint. Repeated hyperparameter selection on validation also introduces selection bias, so the seed analysis should be interpreted as a robustness diagnostic rather than a new independent test.

5. Task 4: Explainability and Failure Analysis

5.1 Methods

We use two simple tools on the final tuned model (threshold 0.613) to understand its decisions. Saliency maps measure how much a tiny change to each pixel would shift the model’s score, showing where it is paying attention, but not in which direction. Occlusion maps cover 8×8 patches with gray and record how the score changes: blue patches push the model toward “real,” red patches push it toward “AI.”

5.2 Looking at the Model’s Worst Mistakes

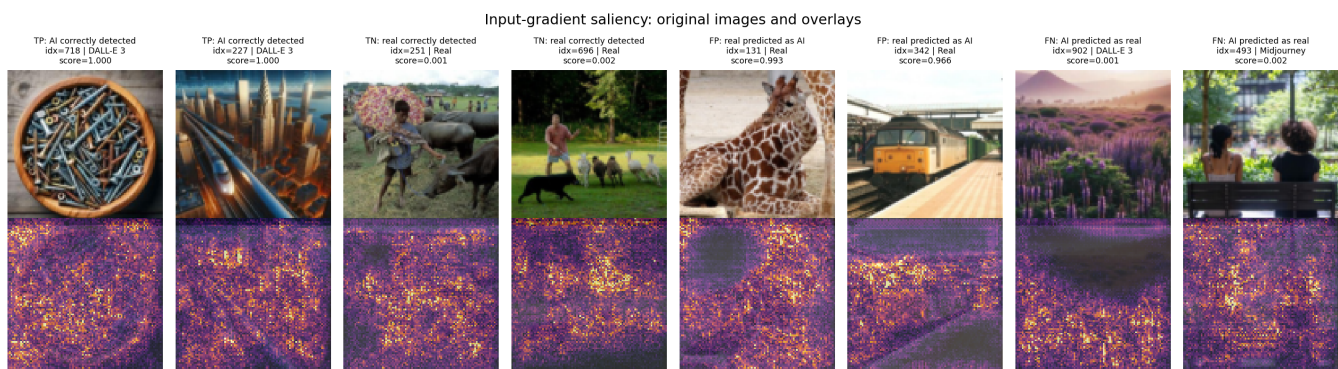


Figure 4: Saliency overlays for confident correct predictions and errors.

Across all eight examples in Figure 4, bright saliency is scattered in small clusters across the whole image rather than on one clear object, suggesting the model combines many small texture and edge cues rather than looking for a single “tell.”

Two failure patterns stand out. The two correctly-detected AI images and two correctly-cleared real images show the same diffuse pattern: no single object or region dominates the saliency map in either case, reinforcing the idea that the model’s decision process itself, correct or not, relies on scattered, low-level evidence rather than one clear signal. A real giraffe photo and a real train photo were both mistaken for AI with high confidence (99.3% and 96.6%). Their occlusion maps (Figure 5) show red patches sitting directly on the giraffe’s spotted coat and the train’s repeating panels, suggesting the model may treat dense, repetitive textures as an AI signal. Conversely, two real-looking AI images were mistaken for real photos with near-total confidence (scores of 0.001 and 0.002); their occlusion maps show only weak, scattered evidence, meaning the model simply never found anything suspicious rather than being actively misled.

Averaging saliency over 100 real and 100 AI images (Figure 6) produces nearly identical patterns for both classes, so the model does not appear to rely on one fixed image location to distinguish them.

5.3 Does the Model Treat All Generators the Same?

The model’s recall is not uniform across the five AI generators it must detect (Table 7). DALL-E 3 and SDXL images are the easiest to catch on clean data (recall 0.9043 and 0.8830), while Midjourney images are consistently the hardest, both before



Figure 5: Occlusion maps for the four confident mistakes above.

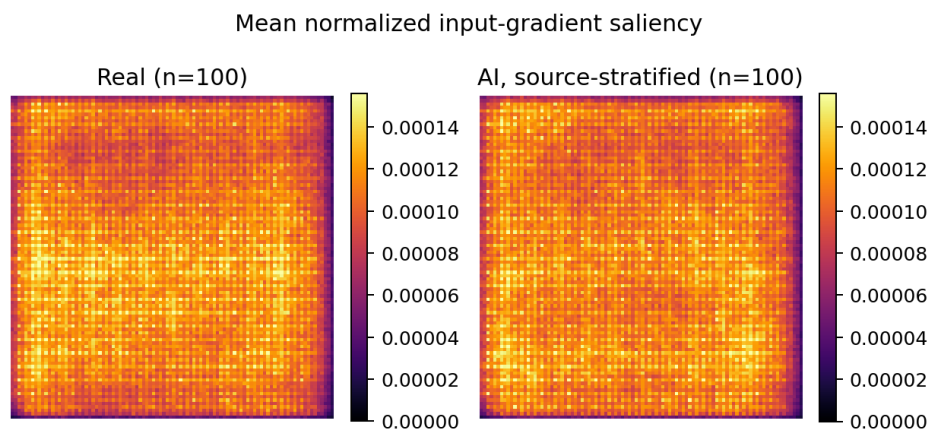


Figure 6: Average saliency across 100 real and 100 AI images.

(0.6452) and after (0.4759) post-processing. Post-processing hurts every generator’s recall, but SD 3 suffers the largest drop (21.1 percentage points). This spread suggests the model has partly learned generator-specific patterns rather than one universal signature of “AI-ness” that works equally well across all generators.

Table 7: Recall by generator on clean and augmented validation.

Generator	Clean recall	Augmented recall	Drop (pp)
SD 2.1	0.7647	0.6277	13.7
SDXL	0.8830	0.6738	20.9
SD 3	0.7166	0.5053	21.1
DALL-E 3	0.9043	0.7540	15.0
Midjourney	0.6452	0.4759	16.9

The score gap between classes also shrinks under post-processing: the mean AI score drops from 0.7865 to 0.6495 while the mean real-image score barely moves (0.2949 to 0.3028). This means post-processing works mainly by making AI images look *less* suspicious, rather than by making real images look *more* suspicious, which matches the recall drop we see without a matching rise in false positives.

5.4 Limitations

Saliency shows sensitivity, not causation, and cannot indicate direction on its own. Occlusion introduces an artificial gray patch, so part of the response may be to that artifact rather than the missing content, and single-patch testing can miss interactions between distant regions. The examples shown are the most confident cases, not typical ones. The generator-level differences and score-gap analysis are aggregate statistics, not visual explanations, so they identify *that* behavior differs by source without fully explaining *why*. Finally, the diffuse pattern we observe is equally consistent with a texture shortcut or with the resize-related geometry issue discussed in Section 2; the two cannot be distinguished from these maps alone. These results should therefore be read as plausible hypotheses, not a proven explanation of the model’s decision rule.

6. Conclusion

We developed a reproducible, CPU-only, Docker-packaged pipeline for AI-image detection under strict time constraints. Dataset exploration revealed strong class-correlated geometry and encoding shortcuts, which we deliberately avoided using as explicit predictive features; conservative cleaning removed only 312 exact duplicate copies from 29 688 training images.

Systems design and automated tuning were both decisive. Early stride-based downsampling, balanced sampling, deadline-driven training, frequent atomic checkpoints, and 30-trial hyperparameter searches produced a final Task 2 model with 0.8098 recall at 0.1755 FPR and 0.8864 AUC. It therefore reaches the suggested 0.8 recall target while respecting the hard FPR limit. The engineered-feature Random Forest remains substantially weaker.

Robust fine-tuning with realistic corruptions and dual-domain calibration achieves 0.6073 recall at 0.1551 FPR on augmented validation and 0.7831 recall at 0.1702 FPR on clean validation. Task 3 therefore also reaches its suggested recall target while satisfying the FPR constraint on both domains. Its main benefit is a safer augmented operating point: relative to Task 2, augmented FPR falls from 0.2620 to 0.1551, although recall and AUC decrease.

The tuning and seed analyses qualify these headline results. Calibration recall transfers more reliably than FPR, and Task 2 lies close enough to both boundaries that only three of ten seeds satisfy both targets simultaneously. Task 3 is more stable on augmented data, with nine of ten seeds reaching 0.6 recall and all ten respecting augmented FPR, although clean FPR exceeds 0.20 for two seeds. The selected seed-0 runs meet all required constraints, but hidden-holdout evaluation remains essential because repeated validation-based tuning and finite-sample threshold calibration can overstate stability.

Explainability suggests diffuse low-level evidence, a plausible sensitivity to repetitive natural textures, and substantial generator-specific differences in both clean and post-processed recall, while also highlighting the limits of sensitivity-based explanations. Future work should tune on a separate development split, enlarge the real calibration set, apply corruptions at native resolution, ablate individual corruption families, directly test the remaining geometry-related shortcut, and rerun explainability on the final tuned checkpoint.